2024aa05257-bhuvangiri Aditya .

Mid_Sem_Report.docx

S2-25_DISSERTATION-NSP4 - Mid Sem

S2-25_DISSERTATION-NSP4

Birla Institute of Technology and Science Pilani WILP Division

Document Details

Submission ID
trn:oid:::1:3596520821

Submission Date
Jun 17, 2026, 10:07 PM GMT+5:30

Download Date
Jun 17, 2026, 10:13 PM GMT+5:30

File Name
Mid_Sem_Report_1_.docx

File Size
203.0 KB

9 Pages

2,165 Words

12,918 Characters

6% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text

Match Groups

14 Not Cited or Quoted 6%
Matches with neither in-text citation nor quotation marks

0 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

4% Internet sources
1% Publications
1% Submitted works (Student Papers)

Match Groups

- **14** Not Cited or Quoted 6%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

- 4% Internet sources
- 1% Publications
- 1% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Student papers	
Birla Institute of Technology and Science Pilani		1%
2	Internet	
ojphi.jmir.org		<1%
3	Internet	
figshare.mq.edu.au		<1%
4	Internet	
www.gabormelli.com		<1%
5	Internet	
www.mdpi.com		<1%
6	Internet	
www.patent-detectives.com		<1%
7	Internet	
www.atlantis-press.com		<1%
8	Internet	
www.freelancermap.com		<1%
9	Publication	
Wenbo Shao, Jun Li, Hong Wang. "Self-Aware Trajectory Prediction for Safe Auton...		<1%
10	Internet	
arxiv.org		<1%

11 Internet

ijpp.gau.ac.ir <1%

AUTOMATED GENERATION OF SUPPORT ASSISTANT TREE

AIMLCZG628T: Dissertation

by

Aditya Bhuvangiri

2024AA05257

1

Dissertation work carried out at

SAP Labs India, Bangalore

Submitted in partial fulfilment of M.Tech. Artificial Intelligence
& Machine Learning degree programme

Under the Supervision of

Akhileswara Kumar Chatala

SAP Labs India, Bangalore

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE

PILANI (RAJASTHAN)

June 2026

ABSTRACT

The Support Assistant is a hierarchical recommendation tree used by SAP customers to navigate to the correct product component when raising a support case. Currently, its creation and maintenance are performed entirely by product experts — a process that takes three to four months for initial onboarding and requires periodic updates every six months. This project proposes and implements a fully automated pipeline for generating the Support Assistant tree from historical incident data.

After generating the recommendations for the Tree generation. The Tree creation consists of four stages. In Stage 1, a reference component hierarchy is constructed using an LLM that analyses the SAP component present in historical recommendations. In Stage 2, recommendations are assigned to tree branches through component-based matching. Stage 3 applies sub-clustering to any branch exceeding seven recommendations, using one of three strategies: K-Means with Sentence-Transformer embeddings, Hierarchical Agglomerative Clustering (HAC) with Sentence-Transformer embeddings, or direct LLM-based semantic clustering. In Stage 4, an LLM generates descriptive branch labels for the newly created sub-clusters.

The three tree-building approaches were evaluated on 111 unique recommendations spanning 15 GRC product components. K-Means and HAC both successfully split overloaded branches and produced balanced trees (Gini coefficient: 0.694), while the LLM method failed to perform splits (Gini: 0.765). HAC emerged as the recommended method, consuming the fewest tokens (868) and completing in 23.3 seconds, while being geometrically better suited to the dense Sentence-Transformer embedding space.

Signature of the Student	Signature of the Supervisor
Name: Aditya Bhuvangiri Date: Place: Bangalore	Name: Akhileswara Kumar Chatala Date: Place: Bangalore

3

CONTENTS

1. INTRODUCTION

The Support Assistant is a core tool within SAP's customer support ecosystem. It is structured as a hierarchical tree where each leaf branch holds a curated set of recommendations relevant to a specific product component. Customers navigate this tree interactively to find resolutions or to raise support cases under the most appropriate component — ensuring accurate routing from the outset.

The current process of building and maintaining this tree is entirely manual, relying on domain experts who must review historical incidents, curate recommendations, and construct the hierarchy from scratch. Initial onboarding requires three to four months, and the tree must be refreshed at least every six months to remain relevant as products evolve.

This dissertation project — Automated Generation of Support Assistant Tree (ASAT) — aims to eliminate this manual bottleneck. The proposed system ingests historical support incident data, extracts recommendations, filters the recommendations and uses a combination of LLMs and clustering algorithms to automatically construct a well-structured, labelled Support Assistant tree. Three distinct tree-building strategies are implemented and compared: K-Means clustering, Hierarchical Agglomerative Clustering (HAC), and direct LLM-based clustering.

As of the mid-semester milestone, the recommendation generation stage is complete and all four stages of the tree construction pipeline have been implemented and evaluated.

2. BACKGROUND AND LITERATURE REVIEW

The problem of automating support tree construction sits at the intersection of Natural Language Processing, clustering, and Large Language Models. The following literature forms the theoretical foundation of the approaches implemented in this project.

- **K-Means:** MacQueen (1967) introduced K-Means clustering — the partition-based approach used in one of the three tree-building strategies. K-Means iteratively assigns data points to the nearest centroid and is computationally efficient for moderate-sized datasets.
- **HAC:** Murtagh (1983) surveyed hierarchical clustering algorithms, establishing HAC with average linkage and cosine distance as a principled approach for grouping semantically similar text representations — directly applicable to the dense embedding vectors produced by Sentence-Transformers.
- **Transformers:** Vaswani et al. (2017) introduced the Transformer architecture underpinning all modern LLMs. The attention mechanism enables contextual understanding of support note text regardless of vocabulary overlap.
- **BERT:** Devlin et al. (2019) introduced BERT, demonstrating that pre-trained bidirectional transformers produce rich sentence representations. This work motivates the use of Sentence-Transformers (all-MiniLM-L6-v2) for embedding support notes.
- **GPT-3:** Brown et al. (2020) demonstrated GPT-3's few-shot capability for structured generation tasks, informing the LLM-based clustering and branch labelling approaches used in Stages 3 and 4.
- **RRF:** Cormack et al. (2009) introduced Reciprocal Rank Fusion (RRF) for combining ranked retrieval results — relevant to the recommendation curation stage that precedes tree construction.

7

3. SYSTEM ARCHITECTURE AND METHODOLOGY

The ASAT pipeline is implemented as a four-stage agentic system. Each stage is modular and independently testable. The three tree-building strategies (K-Means, HAC, LLM) differ at Stage 3; all other stages are identical across the three approaches. This design enables fair comparison of the clustering strategies in isolation.

The input to for the Tree generation is filtered recomendation — a curated set of 111 unique recommendations across 15 SAP GRC-SAC (SAP Access Control) product components, derived from historical customer support incidents. The recommendation generation stage produced this dataset using NLP-based trending topic extraction and ranking.

3.1 Stage 1 – Reference Tree Construction (LLM)

A reference component hierarchy is constructed by bringing out the product components present in the filtered recommendations and an algorithm is devised to compare and hierarchically build a Reference_Tree.

The Application component hierarchy(ACH) collection is used to label the branches based on the component naming.

This stage returns a JSON tree whose structure mirrors the component naming convention: for example, GRC-SAC-EAM is placed as a child of GRC-SAC, which is itself a child of GRC.

3.2 Stage 2 – Note Assignment

Each recommendation row in the dataset contains a component. This stage assigns every recommendation to the deepest matching node in the reference tree using exact component matching. If no exact match exists, the algorithm progressively drops the last segment of the component code (e.g. GRC-SAC-ARQ-SF falls back to GRC-SAC-ARQ) until a match is found.

3.3 Stage 3 – Sub-Clustering (Three Methods)

Any leaf node that accumulates more than seven recommendations is sub-clustered to maintain branch quality. Three interchangeable strategies are implemented, selectable at runtime:

- **Method 1 – K-Means:** Sentence-Transformer embeddings (all-MiniLM-L6-v2, 384 dimensions) are computed for each note's title and phrases. K-Means is then applied, with the optimal k chosen by maximising the silhouette score across k=2 to k=MAX_CLUSTERS (default 5). This method is fast and deterministic.
- **Method 2 – HAC:** The same Sentence-Transformer embeddings are used with Hierarchical Agglomerative Clustering using average linkage and cosine distance. Cosine distance is geometrically more appropriate than Euclidean distance for unit-normed embedding vectors, giving HAC a theoretical advantage over K-Means for this embedding space. The optimal cluster count is again selected by silhouette score.
- **Method 3 – LLM Clustering:** The LLM is provided with the note numbers, titles, and phrases for each overloaded leaf and asked to return a JSON cluster assignment. This method leverages the LLM's semantic understanding of SAP GRC domain vocabulary but introduces non-determinism and higher token cost.

3.4 Stage 4 – Automated Branch Labelling

For each new sub-cluster branch created in Stage 3, the LLM generates a short descriptive label (3-6 words) based on the titles and phrases of the recommendations it contains. This stage runs only when Stage 3 produces new branches; if no branches are created, Stage 4 is skipped entirely.

4. IMPLEMENTATION DETAILS

The Automated Tree generation is implemented in Python 3.11 and structured as a set of independent modules, each corresponding as a stage. An orchestrator coordinates the stage execution and saves the final tree.

Module	Responsibility
models.py	TreeNode dataclass, ID counter, JSON loaders
tree_builder.py	LLM prompt construction and reference tree parsing
note_assignment.py	Component-based note assignment and metadata builder
clustering.py	HAC clustering along with Sentence-Transformer embedding
title_generator.py	LLM-based cluster branch title generation
main.py	Automated Tree Generation orchestrator with CLI argument handling

Key technology choices:

- Sentence-Transformers all-MiniLM-L6-v2 – 384-dim dense embeddings for note text
- claude-4.6-sonnet via SAP AI Core - Used for stage 4 -> Branch labelling
- scikit-learn – KMeans, AgglomerativeClustering, silhouette_score

5. EXPERIMENTAL RESULTS AND EVALUATION

All three tree-building approaches were run on the same input dataset (111 unique recommendations, 15 components, cluster threshold = 7).

Metric	K-Means	HAC	LLM
Total nodes	21	21	19
Total leaves	14	14	13
Max depth	4	4	4
Avg leaf depth	3.29	3.29	3.15
Leaves split (Stage 3)	1	1	0
Leaf notes – max	7	7	23
Leaf notes – mean	2.57	2.57	2.77
Leaf notes – std dev	4.13	4.13	6.23
Balance score (Gini)	0.6944	0.6944	0.765
Component coverage	100%	100%	100%
Total LLM calls	3	3	2
Total tokens	868	868	1497
Clustering(Stage 3) duration	10.93s	9.14s	8.27s
Total pipeline duration	23.66s	23.34s	17.82s*

6. COMPARATIVE ANALYSIS OF TREE-BUILDING METHODS

6.1 Pairwise Spearman Rank Correlation

Spearman rank correlation, which is sensitive to ordinal differences, reveals a divergence: K-Means vs HAC achieves rho = 1.0000, confirming identical rank ordering. Both drop to rho = 0.9632 when compared against LLM, driven by LLM's divergence on leaf_notes_max (23 vs 14), leaf_notes_std (6.23 vs 4.13), and leaves_split (0 vs 1).

6.2 Balance Analysis

The Gini coefficient measures how evenly notes are distributed across leaf nodes (0 = perfectly even, 1 = all notes in one leaf).

Method	Gini Score	Rank	Notes
K-Means	0.6944	Joint best	1 leaf split, 14 total leaves
HAC	0.6944	Joint best	1 leaf split, 14 total leaves
LLM	0.7650	Worst	0 leaves split; one 23-note catch-all leaf

LLM's Gini score is 10.2% worse than the embedding-based methods, entirely because the LLM failed to split the one leaf exceeding the threshold during this run.

6.3 Cost and Token Analysis

Token counts reveal that LLM clustering consumes 72% more tokens than either embedding method (1497 vs 868), yet produces inferior tree quality. The embedding-based methods incur no Stage 3 token cost at all — only Stage 4 title generation uses tokens.

Method	Stage 3(Clustering) Tokens	Stage 4(Branch Labelling) Tokens	Total	Stage 3(Clustering) Time
K-Means	0	868	868	10.93s
HAC	0	868	868	9.14s
LLM	1497	0	1497	8.27s

6.4 Recommendation

HAC with Sentence-Transformer embeddings is the recommended method for the following reasons:

- **Geometric correctness:** Sentence-Transformer produces unit-normed 384-dim vectors. Cosine distance is the geometrically correct metric for this space. HAC uses cosine distance natively; K-Means uses Euclidean distance which is less appropriate on a unit hypersphere.
- **Hierarchical affinity:** HAC builds a dendrogram that naturally reflects the hierarchical structure of SAP component notes, aligning with the tree-building objective.
- **Reliability:** Both embedding methods succeed where LLM fails: they correctly split overloaded leaves and achieve a better balance score (0.694 vs 0.765).
- **Speed:** HAC matches K-Means on tree quality while running Stage 3 in 9.14s vs 10.93s — a 16% speedup.

7. FUTURE PLAN

Sl. No.	Phase	Start – End Date	Work to be Done	Status
1	Dissertation Outline	25 Apr – 10 May 2026	Literature review and prepare Dissertation Outline	COMPLETED
2	Design, Development & Mid Sem Report	11 May – 15 Jun 2026	Design, development activity and Mid Sem Report Generation	COMPLETED
3	Testing and Final Report	16 Jun – 17 Jul 2026	Software testing, user evaluation and conclusion	IN PROGRESS
4	Dissertation Review	17 Jul – 27 Jul 2026	Submit dissertation to Supervisor and Additional Examiner for review and feedback	PENDING
5	Final Review and Submission	28 Jul – 2 Aug 2026	Final review and submission of Dissertation	PENDING

8. ABBREVIATIONS

Abbreviation	Full Form
ASAT	Automated Generation of Support Assistant Tree
GRC	Governance, Risk and Compliance
HAC	Hierarchical Agglomerative Clustering
LLM	Large Language Model
NLP	Natural Language Processing
EAM	Emergency Access Management
ARA	Access Risk Analysis
ARQ	Access Request
UAR	User Access Review
CV	Coefficient of Variation
RRF	Reciprocal Rank Fusion
BERT	Bidirectional Encoder Representations from Transformers
GPT	Generative Pre-trained Transformer

9. REFERENCES

1. J. MacQueen. Some Methods for Classification and Analysis of Multivariate Observations. In Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability, Vol. 1, pp. 281-297, 1967.
2. F. Murtagh. A Survey of Recent Advances in Hierarchical Clustering Algorithms. The Computer Journal, 26(4), pp. 354-359, 1983.
3. A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention Is All You Need. Advances in Neural Information Processing Systems (NeurIPS), 2017.
4. J. Devlin, M. Chang, K. Lee, and K. Toutanova. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL-HLT, 2019.
5. T. Brown et al. Language Models are Few-Shot Learners (GPT-3). Advances in Neural Information Processing Systems (NeurIPS), 2020.
6. G. V. Cormack, C. L. A. Clarke, and S. Buttcher. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. Proceedings of the 32nd International ACM SIGIR Conference, pp. 758-759, 2009.
7. N. Reimers and I. Gurevych. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. Proceedings of EMNLP-IJCNLP, 2019.